AR1 Simulation Comparison

n=500, m=1000, AR1 ρ=0.3, σ_s=0.3, 100 reps per F_{ST}

Main points

- R's free-intercept slope (`res_reg_free`) and Python's LDSC estimators (`h2_ldsc_nw`, `h2_ldsc_fixed_nw`) are **different estimators**, not bugs: they differ in intercept constraint, LD-score source, and bias correction.
- **Bias correction explains most of the fixed-intercept gap** in the in-sample case. Python's default LD-score correction is a near-constant shift — it barely moves the free-intercept slope (cancelled by centering) but closes most of the fixed-intercept mismatch at low-to-moderate F_{ST}.
- **A reference panel only works if it actually matches the sample's stratification.** With an independently-redrawn stratification vector f, the free-intercept estimator collapses toward zero on *both* R and Python — confirmed down to a correlation of ≈0 between the reference panel's LD score and the sample's χ^2 statistic, even at F_{ST}=0 (no stratification at all) and even after scaling n and m up 4×.
- **Sharing the same f between the reference panel and the sample recovers the signal.** This isolates the mechanism precisely: a reference panel is only informative to the extent it matches the sample on the specific thing driving each SNP's LD-score magnitude. Once shared, both free- and fixed-intercept estimators recover a sensible shape in both languages.
- **Bias correction matters a lot for fixed intercept, in the reference-panel case too — not just in-sample.** Removing R's bias correction from the reference-panel LD score (while Python keeps its own) blows the R-vs-Python gap out from ±0.13 to as much as −1.04 at low F_{ST}. Free intercept is still essentially unaffected either way (differences under 0.01), exactly as the centering-cancels-a-constant-shift math predicts.
- A residual gap (∼0.02–0.13, fixed intercept) remains even with bias correction matched on both sides — pointing to something inside Python's IRWLS/jackknife regression machinery, not the LD score itself.

Section 1: In-sample LD scores

Both R and Python recompute the LD score from each replicate's own data every time (no reference panel). R's columns are uncorrected; Python applies its default bias correction.

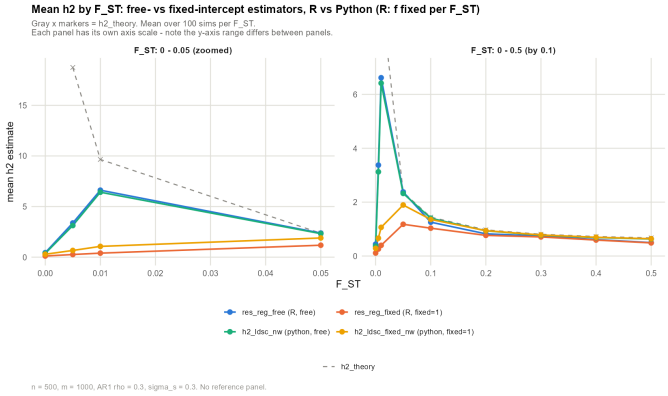


Figure 1. Stratification vector f fixed once per F_{ST}, shared across all 100 reps.

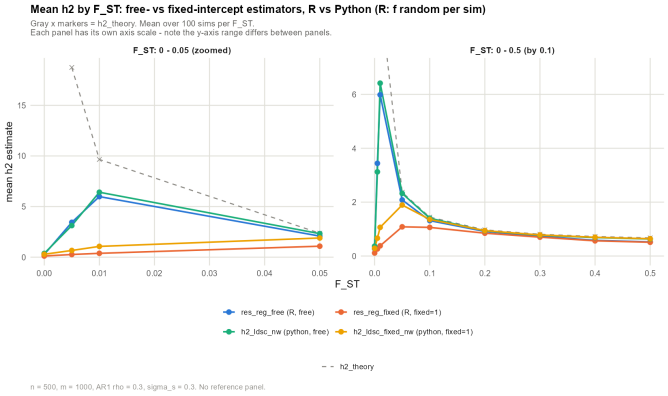


Figure 2. Stratification vector f redrawn fresh every replicate (matches Python's generative design; this is what makes *h2_theory* agree between R and Python).

Section 2: Reference-panel LD scores, R's original (uncorrected) implementation

Both sides use a separately-simulated reference panel (same generative process, independent individuals) for the LD score, reused across all 100 replicates. This is R's *original* implementation, before bias correction was added to the R code — R's reference-panel LD score has no correction term here, while Python's still applies its default bias correction. The only thing that changes between the two figures below is whether the reference panel's stratification draw f is the *same* one used to generate the 100 main replicates, or an independent one.

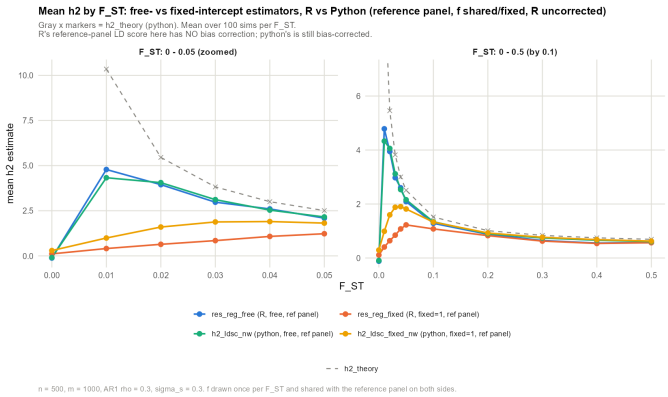


Figure 3. *f* shared, R's reference LD score uncorrected. Free intercept recovers a sensible shape on both sides; fixed intercept has R sitting below Python.

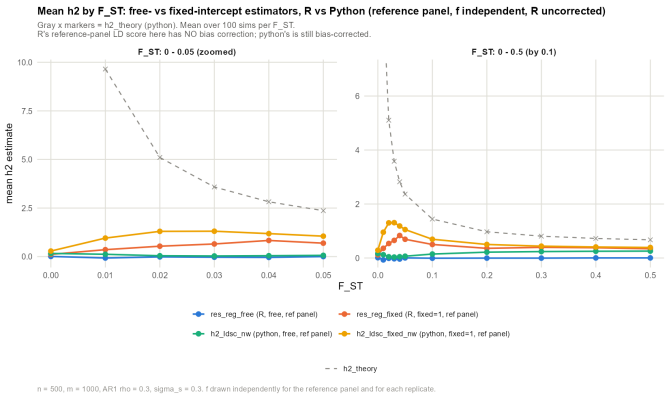


Figure 4. *f* independent, R's reference LD score uncorrected. Free intercept collapses toward zero on both R and Python; fixed intercept shows the widest R-vs-Python gap in the report.

Section 3: Adding bias correction to R's reference-panel LD score

Same two configurations as Section 2 (f shared / f independent), but now R's reference-panel LD score gets the same bias-correction term Python already applies. Fixed intercept moves substantially closer to Python; free intercept is essentially unchanged either way.

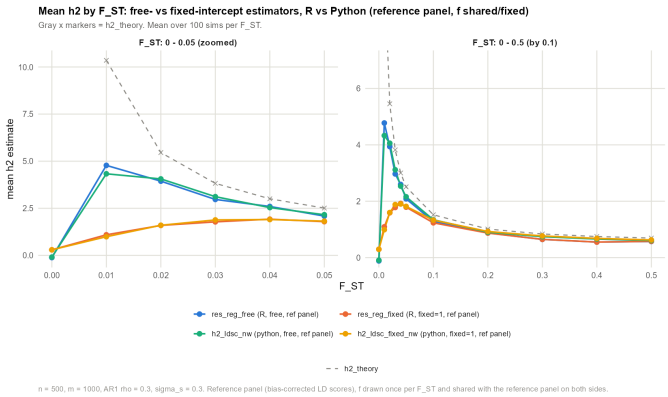


Figure 5. *f* shared, R's reference LD score bias-corrected (matches Python's correction). Fixed intercept now tracks Python closely.

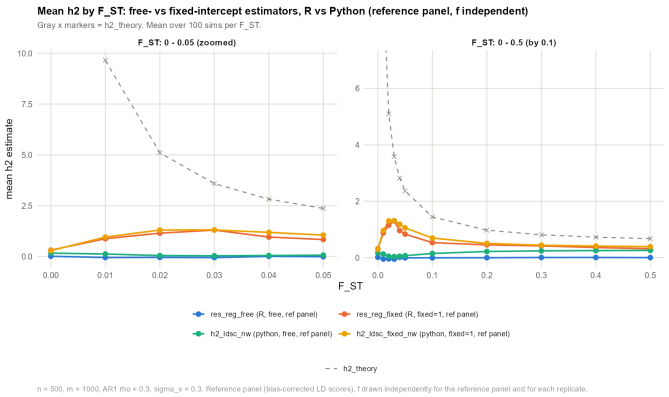


Figure 6. *f* independent, R's reference LD score bias-corrected. Free intercept still collapses toward zero on both sides — bias correction doesn't touch that mechanism.

F_{ST}	Python (fixed)	R, uncorrected (Fig. 3–4)	R, bias-corrected (Fig. 5–6)	diff (R_{nobc} − py)	diff (R_{bc} − py)
0.00	0.294	0.111	0.295	−0.183	+0.001
0.01	0.990	0.407	1.092	−0.583	+0.102
0.02	1.598	0.641	1.590	−0.958	−0.008
0.03	1.881	0.846	1.779	−1.035	−0.102
0.04	1.903	1.078	1.922	−0.825	+0.019
0.05	1.816	1.227	1.789	−0.589	−0.027
0.10	1.330	1.078	1.234	−0.252	−0.096
0.50	0.626	0.567	0.576	−0.059	−0.050

What drove each gap

Factor	Effect on free intercept	Effect on fixed intercept
Intercept constraint (free vs. fixed=1)	—	Not the dominant cause once other factors are matched
Bias correction (near-constant LD-score shift), in-sample	None (cancelled by centering)	Closes most of the in-sample gap at low–mid F_{ST}
Bias correction, reference panel (Figures 3–6)	None (diffs <0.01 regardless of f-sharing)	Large: removing it alone widens the R-vs-Python gap to −1.04 at low F_{ST}
Reference panel with independent f (Figures 4, 6)	Collapses toward noise (both languages) — confirmed at F_{ST}=0 and up to 4× larger n,m	Degraded but still informative
Reference panel with shared f (Figures 3, 5)	Recovers a sensible shape in both languages	Recovers a sensible shape in both languages
Python's IRWLS/jackknife internals	Still unexplained	Best remaining candidate for the ∼0.02–0.13 residual

Open question: whether Python's `do_ldsc_regression` (weights=1, twostep=None path) is exactly mathematically identical to R's closed-form slope, or introduces a small systematic offset — not yet traced through.